

Introduction to Bash Scripting

What is Script?

- A **script** in Linux is a series of commands stored in a file that the shell (e.g., Bash) executes sequentially.
- Scripts help automate repetitive tasks, making them easier to execute without manually typing each command.
- Bash scripts are the most common form of scripting in Linux.

Viewing the Current User's Bash Information in Linux

In Linux, **Bash** (Bourne Again Shell) is the default shell for many Linux distributions. You can check the current user's shell in several ways:

1. Using the `echo` Command

To see the current shell being used:

```
echo $SHELL
```

This will output something like `/bin/bash`, which indicates that the current shell is Bash.

2. Using the `whoami` Command

The `whoami` command shows the currently logged-in user:

```
whoami
```

This will display the username of the current session.

3. Using the `/etc/passwd` File

You can check the default shell for the current user by looking at the `/etc/passwd` file:

```
grep $USER /etc/passwd
```

This command shows the user's information, including their default shell. The last field in the output will indicate whether it is `/bin/bash` or another shell.

The University of Lahore

4. Using the `ps` Command

To see the currently running shell (if different from the default):

```
ps -p $$
```

This shows the process associated with the current shell.

5. Using the `chsh` Command

You can see or change the current user's default shell using:

```
chsh -l
```

First Script

Here's a **step-by-step procedure** to create and run your first Bash script, explained in a way a layperson can follow:

1. Open the Terminal

- On most Linux systems, you can open the terminal by pressing `Ctrl + Alt + T`, or searching for "Terminal" in your application menu.

2. Create a New File

- To create a script, you first need to create a file where you will write the commands.
- In the terminal, use the `touch` command to create a new file. Let's name the file `first_script.sh` (you can choose any name, but it's common to use `.sh` extension for Bash scripts).

```
touch first_script.sh
```

This command creates a new, empty file named `first_script.sh`.

3. Open the File in a Text Editor

- Now, you need to open the file to write your script. You can use any text editor like `nano`, `vim`, or a graphical editor like `gedit` for this.
- Let's use `nano`, a simple terminal-based text editor:

The University of Lahore

```
nano first_script.sh
```

4. Write Your First Script

- Inside the nano editor (or whichever editor you opened), start writing your first script. A simple script might look like this:

```
#!/bin/bash  
echo "Hello, World!"
```

Explanation:

- ❖ **#!/bin/bash:** This is called the **shebang**. It tells the system to use Bash to interpret this script.
- ❖ **echo "Hello, World!":** The echo command prints text to the terminal. In this case, it prints "Hello, World!".

5. Save and Exit the Editor

- After writing the script, save the file and exit the editor:
 - ❖ In nano: Press Ctrl + O to save, then press Enter to confirm. Press Ctrl + X to exit.
 - ❖ In vim: Press Esc, then type :wq and press Enter.

6. Make the Script Executable

- Before you can run your script, you need to give it executable permission. This tells the system that you want to treat the file as a program that can be run.
- To make the script executable, use the chmod command:

```
chmod +x first_script.sh
```

7. Run the Script

- Now that the script is executable, you can run it. To do this, use the following command:

```
./first_script.sh
```

- ❖ The ./ tells the system to look for the script in the current directory.

The University of Lahore**8. See the Output**

- After running the script, you should see the following output in the terminal:

**Comments in Batch Scripting**

- **Comments** in Linux **Bash scripts** are lines or parts of lines that are ignored by the Bash interpreter.
- They are used to explain code, add notes for the developers, or disable code for testing purposes.
- Comments are important for making scripts more understandable and maintainable, especially when scripts grow in complexity or are shared among multiple developers.

Creating Single-Line Comments in Bash

- In **Bash**, single-line comments start with the # symbol.
- Everything after the # on that line is ignored by the interpreter.

Example of Single-Line Comment:

```
#!/bin/bash  
  
# This is a single-line comment  
echo "Hello, World!" # This prints Hello, World! to the terminal
```

- In this example, both # This is a single-line comment and # This prints Hello, World! to the terminal are ignored when the script runs.

Creating Multi-Line Comments in Bash

- Bash does not have a specific syntax for **multi-line comments** like some programming languages, but there are two common ways to achieve the same effect.

1. Using Multiple Single-Line Comments

The most straightforward way to create multi-line comments in Bash is to use multiple # symbols at the beginning of each line.

The University of Lahore**Example:**

```
#!/bin/bash

# This is a multi-line comment
# It explains the purpose of the script
# and provides additional context for the user.
echo "Hello, World!"
```

- Each line of the multi-line comment starts with a #.

2. Using a Here Document for Multi-Line Comments

This method uses the << syntax to define a block of text that the interpreter will ignore.

Example:

```
#!/bin/bash
: << 'END'
This is a multi-line comment.
You can write as many lines as you need here.
None of this text will be executed or displayed.
END
echo "Hello, World!"
```

- : << 'END' starts the multi-line comment block.
- END marks the end of the comment block.
- The colon (:) is a no-op command that does nothing, and the << 'END' redirects the text block to nowhere, effectively ignoring it.
- You can write any word as the replacement of END but it should not be keyword.

Advantages of Using Comments:

- **Clarity:** Explain complex sections of code or why certain decisions were made.
- **Maintenance:** Makes scripts easier to maintain and debug.
- **Collaboration:** Helps others (or yourself in the future) understand your script more easily.

The University of Lahore

Variables in Batch Scripting

- In **Bash scripting**, variables are used to store values that can be referenced and manipulated within the script.
- They make the script more dynamic and flexible by allowing the same code to work with different inputs.
- Variables in Bash can store strings, numbers, filenames, and more.

Declaring Variables

- In Bash, you declare a variable simply by writing its name and assigning it a value using the = sign.
- There is **no space** between the variable name, the equal sign, and the value.

```
#!/bin/bash
name="John Doe" # Declaring a string variable
age=30          # Declaring a numeric variable
echo "Name: $name"
echo "Age: $age"
```

Important Note:

Unlike many other languages, you don't need to specify the data type of the variable in Bash. Everything is treated as a string unless explicitly treated otherwise.

Dollar Sign \$:

To reference a variable's value, use the dollar sign (\$) followed by the variable's name.

Variable Naming Rules:

- Names must start with a letter or an underscore (_).
- They can contain letters, numbers, and underscores.
- Avoid using spaces around the = sign.
- Variables are case-sensitive (`name` and `NAME` are different).

The University of Lahore**Types of Variables****a) Local Variables**

These are variables declared within a script or function and are only available within that specific script or function.

```
#!/bin/bash
local_var="I'm local"
echo $local_var # Output: I'm local
```

b) Environment Variables

These are variables that are available to all programs running in the shell. You can make a variable an environment variable by using the **export** command.

```
#!/bin/bash
export GLOBAL_VAR="I am global"
echo $GLOBAL_VAR
```

Important Point

If you want to use global variable in another script you must run the that script which has global variable using the following command

```
source scripts1.sh (script which has global variable)
```

then run that script which using the global variable.

c) Special Variables

Bash provides several special variables that are automatically set by the shell. Some commonly used ones include:

- **\$0**: The name of the script.
- **\$1, \$2, ...**: Positional parameters representing arguments passed to the script.
- **\$#**: The number of arguments passed to the script.
- **\$@**: All arguments passed to the script.
- **\$?**: The exit status of the last command executed.
- **\$\$**: The process ID of the current script.
- **\$USER**: The current logged-in username.

The University of Lahore**Working with Variables****a) Using Variables in Strings**

- To reference a variable in a string, use double quotes " around the string, so the variable gets expanded.
- Single quotes ' will treat the variable name as a literal string and not expand it.

```
#!/bin/bash
name="John"
echo "Hello, $name!"      # Output: Hello, John!
echo 'Hello, $name!'      # Output: Hello, $name!
```

b) Command Substitution

You can assign the output of a command to a variable using `$(command)` or backticks ``command``.

```
#!/bin/bash
date_now=$(date)  # Using $(...)
echo "Current date and time: $date_now"
current_user=`whoami`  # Using backticks
echo "Current user: $current_user"
```

c) Arithmetic Operations

Bash handles arithmetic operations using `(( ))` or `$( ( ))` syntax.

```
#!/bin/bash
a=5
b=3
sum=$((a + b))  # Addition
diff=$((a - b))  # Subtraction
prod=$((a * b))  # Multiplication
quot=$((a / b))  # Division
echo "Sum: $sum"
echo "Difference: $diff"
echo "Product: $prod"
echo "Quotient: $quot"
```


The University of Lahore

d) Using Variables with read

You can get user input and store it in a variable using the `read` command.

```
#!/bin/bash
echo "Enter your name:"
read name
echo "Hello, $name!"
```

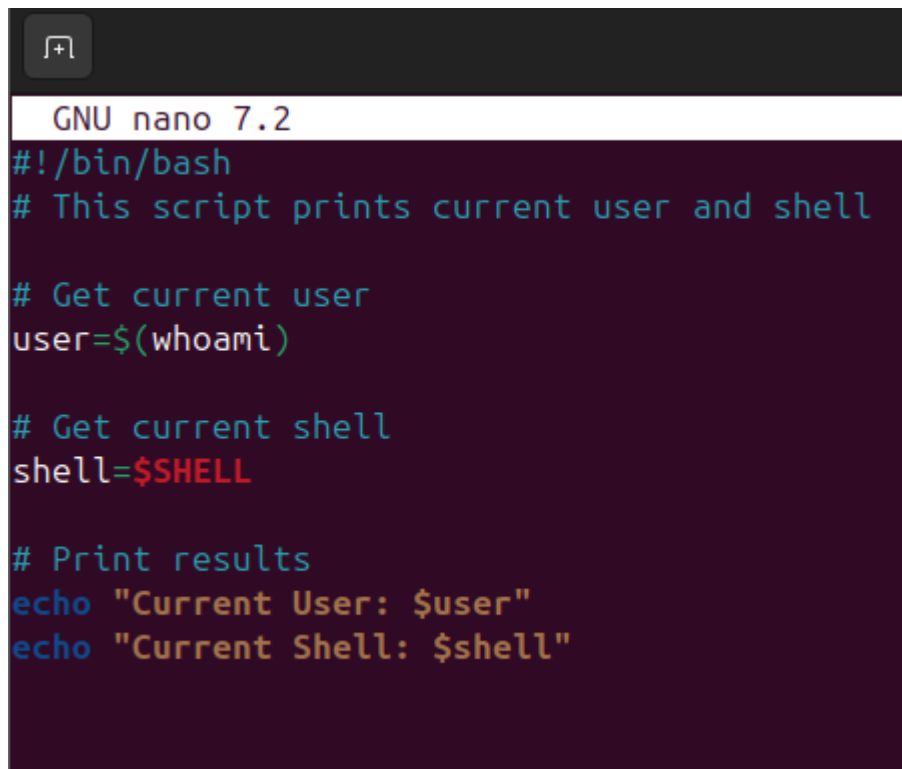
e) Default Values Using \${variable:-default}

You can set a default value for a variable if it is not set using `${var:-default}`.

```
#!/bin/bash
name=${name:-"Unknown"}
echo "Name: $name"
```

Practice Questions

- 1) Write a Bash script named `user_info.sh` that prints the current user's name and the current shell being used. Use comments to explain each step in your script.



```
GNU nano 7.2
#!/bin/bash
# This script prints current user and shell

# Get current user
user=$(whoami)

# Get current shell
shell=$SHELL

# Print results
echo "Current User: $user"
echo "Current Shell: $shell"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano user_info.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x user_info.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./user_info.sh
Current User: abdullah-sohail
Current Shell: /bin/bash
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

The University of Lahore

- 2) Create a script named `system_info.sh` that retrieves and prints the current date and the logged-in users. Use comments to describe what each command does.

```
GNU nano 7.2
#!/bin/bash
# This script shows date and logged-in users

# Get current date
current_date=$(date)

# Get logged-in users
users=$(who)

# Print info
echo "Current Date: $current_date"
echo "Logged-in Users:"
echo "$users"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano system_info.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x system_info.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./system_info.sh
Current Date: Tue Mar 31 04:54:40 PM PKT 2026
Logged-in Users:
abdullah-sohail seat0      2026-03-31 16:48 (login screen)
abdullah-sohail tty2      2026-03-31 16:48 (tty2)
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 3) Write a script called `disk_usage.sh` that stores the output of the `df -h` command (which displays disk usage) in a variable and prints it. Include comments explaining each step.

```
GNU nano 7.2
#!/bin/bash
# This script stores disk usage in a variable

# Store df -h output
disk=$(df -h)

# Print disk usage
echo "Disk Usage:"
echo "$disk"
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano disk_usage.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x disk_usage.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./disk_usage.sh
Disk Usage:
Filesystem      Size  Used Avail Use% Mounted on
tmpfs            286M  1.7M  284M   1% /run
/dev/sda2        30G   11G   18G  39% /
tmpfs            1.4G   0    1.4G   0% /dev/shm
tmpfs            5.0M   8.0K  5.0M   1% /run/lock
tmpfs            286M  120K  286M   1% /run/user/1000
/dev/sr0          96M   96M    0 100% /media/abdullah-sohail/CDROM
/dev/sr1         6.0G   6.0G    0 100% /media/abdullah-sohail/Ubuntu 24.04.3 LTS amd64
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 4) Create a script named `greet_user.sh` that prompts for a username. If no input is given, it should default to "Guest". Print a greeting message with the user's name. Use comments to explain the script's functionality.

```
GNU nano 7.2
#!/bin/bash
# This script greets user

# Ask for username
read -p "Enter your name: " name

# Check if empty
if [ -z "$name" ]; then
    name="Guest"
fi

# Print greeting
echo "Hello, $name!"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano greet_user.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x greet_user.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./greet_user.sh
Enter your name: Abdullah
Hello, Abdullah!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 5) Write a script named `sum_numbers.sh` that prompts the user for two numbers, calculates the sum using variables, and prints the result. Include comments for clarity.

The University of Lahore

```
GNU nano 7.2
#!/bin/bash
# This script adds two numbers

# Take input
read -p "Enter first number: " num1
read -p "Enter second number: " num2

# Calculate sum
sum=$((num1 + num2))

# Print result
echo "Sum = $sum"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano sum_numbers.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x sum_numbers.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./sum_numbers.sh
Enter first number: 12
Enter second number: 12
Sum = 24
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 6) Create a script called list_files.sh that stores the list of files in the current directory in a variable and prints it. Use comments to explain the script's operations.

```
GNU nano 7.2
#!/bin/bash
# This script lists files

# Store files
files=$(ls)

# Print files
echo "Files in current directory:"
echo "$files"
```

The University of Lahore

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano list_files.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x list_files.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./list_files.sh
Files in current directory:
disk_usage.sh
greet_user.sh
list_files.sh
sum_numbers.sh
system_info.sh
user_info.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 7) Write a script named env_var.sh that checks for an environment variable named MY_VAR. If it is not set, assign it a default value of "Default Value". Print the value of MY_VAR. Include comments to clarify the code.

```
GNU nano 7.2
#!/bin/bash
# This script checks environment variable

# Check if MY_VAR is empty
if [ -z "$MY_VAR" ]; then
    MY_VAR="Default Value"
fi

# Print variable
echo "MY_VAR = $MY_VAR"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano env_var.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x env_var.sh
./env_var.sh
MY_VAR = Default Value
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 8) Write a script named dir_size.sh that calculates the size of the current directory and prints it. Store the output of the du -sh command in a variable. Include comments for clarity.

```
GNU nano 7.2
#!/bin/bash
# This script calculates directory size

# Store size
size=$(du -sh .)

# Print result
echo "Directory Size:"
echo "$size"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano dir_size.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x dir_size.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./dir_size.sh
Directory Size:
36K .
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

- 9) Create a script called create_file.sh that prompts the user for a filename. If no filename is provided, it should default to "default.txt". The script should create an empty file with the specified name and print a confirmation message. Include comments to explain each par.

```
GNU nano 7.2
#!/bin/bash
# This script creates a file

# Ask filename
read -p "Enter filename: " filename

# Default if empty
if [ -z "$filename" ]; then
    filename="default.txt"
fi

# Create file
touch "$filename"

# Confirmation
echo "File '$filename' created successfully!"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano create_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x create_file.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./create_file.sh
Enter filename: Demo1.sh
File 'Demo1.sh' created successfully!
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```

The University of Lahore

Challenge Question

- 10) Write a script file_logger.sh that prompts the user to create a file. After the file is created, the script should log the creation time and filename into a log.txt file.

```
GNU nano 7.2
#!/bin/bash
# This script creates a file and logs the creation time

# Ask user for filename
read -p "Enter filename: " filename

# Check if input is empty
if [ -z "$filename" ]; then
    echo "No filename provided. Exiting..."
    exit 1
fi

# Create the file
touch "$filename"

# Get current date and time
current_time=$(date)

# Log info into log.txt
echo "File '$filename' created at $current_time" >> log.txt

# Confirmation message
echo "File '$filename' created successfully!"
echo "Log updated in log.txt"
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ nano file_logger.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ chmod +x file_logger.sh
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ ./file_logger.sh
Enter filename: textfile.txt
File 'textfile.txt' created successfully!
Log updated in log.txt
```

```
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$ cat log.txt
File 'textfile.txt' created at Tue Mar 31 05:11:40 PM PKT 2026
abdullah-sohail@abdullah-sohail-VMware-Virtual-Platform:~/bash_lab$
```